

AD-WM: Action-Discriminative World Models for Counterfactual Model Predictive Control

Anonymous Authors

Abstract—Latent world models are typically trained to predict factual transitions, whereas model predictive control (MPC) must compare alternative actions from the same state. A model can therefore achieve low factual prediction error yet poorly distinguish candidate actions. We introduce AD-WM, an action-discriminative joint-embedding world model for counterfactual MPC. AD-WM combines residual latent dynamics with predictor-level action-recovery regularization, using inverse dynamics and a normalized recovery objective motivated by conditional mutual information. Both objectives encourage planning transitions to preserve action information; their auxiliary heads are discarded at test time, leaving MPC unchanged. On OGBench-Cube, AD-WM improves hard-start success from 3.7% to 52.0% over a matched LeWM baseline and improves mean success over the reproduced baseline in four of five simulation environments. Planning diagnostics show that factual prediction error and whole-bank action ranking do not follow the closed-loop success ordering, whereas CEM-aligned elite regret tracks success more closely. With a frozen V-JEPA 2 encoder and matched DROID post-training, AD-WM also improves zero-shot transfer to our Franka setup, increasing basic pick-and-place success from 42.2% to 71.1% without lab-specific adaptation. These results suggest that world models for planning should preserve action-dependent differences needed for counterfactual selection, rather than optimize factual prediction accuracy alone. More demonstration videos are available on our anonymous project website: <https://ad-wm.github.io/>.

I. INTRODUCTION

World models enable agents to predict the consequences of candidate actions before executing them. For vision-based control, methods such as PlaNet, Dreamer, and TD-MPC learn latent dynamics from visual observations and use predicted trajectories to guide action selection [1]–[6]. More recently, joint-embedding predictive architectures (JEPAs) have enabled future representation prediction without pixel reconstruction [7], [8]. Building on this approach, DINO-WM, PLDM, LeWorldModel (LeWM), and V-JEPA 2-AC use predicted features to plan toward visual goals [9]–[12]. These methods evaluate candidate action sequences through latent rollouts and select actions whose predicted outcomes approach the goal.

Accurate prediction of recorded transitions, however, does not guarantee useful comparisons between alternative actions. Training supervises the outcome of the action actually taken, whereas planning compares what would happen under different actions from the same state. Fig. 1 illustrates this distinction through a cube-grasping example. When visual representations are dominated by persistent scene content, a predictor can achieve low error by largely preserving the current representation. Yet its predictions may obscure the smaller action-dependent changes that distinguish ap-

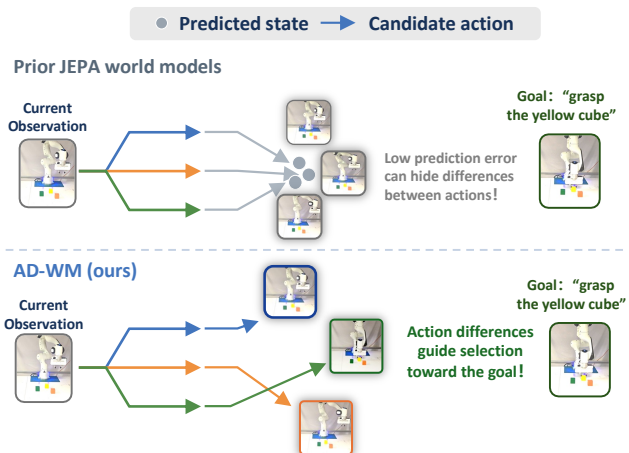


Fig. 1: **Action discrimination for planning.** Low prediction error can mask differences between candidate actions. AD-WM preserves action-dependent changes to guide goal-directed selection.

proaching the cube from moving away. Such predictions fit observed transitions but provide limited guidance for goal-directed action selection.

This motivates a key requirement for planning: *latent dynamics should preserve the action-dependent differences needed for counterfactual comparison*. Factual supervision anchors predictions to observed outcomes but does not explicitly enforce action recoverability. Recovering the action from the current and predicted next representations provides a complementary training signal. Applied to model-generated transitions, this constraint directly shapes the dynamics that MPC uses to compare candidate actions. We therefore focus on *predictor-level* action discrimination.

We propose **AD-WM**, which combines residual latent prediction with predictor-level action recovery for counterfactual MPC. The residual predictor estimates latent increments relative to the current state. Action recovery uses two related objectives: inverse dynamics and normalized recovery motivated by conditional mutual information. By default, both operate on the current and predicted next representations. In simulation, the encoder and predictor are trained jointly. At deployment, the auxiliary heads are discarded, leaving MPC unchanged.

We also examine whether the learned dynamics support useful candidate selection. The cross-entropy method (CEM) retains a small set of high-scoring action sequences, called the elite set, to guide subsequent search [13]. A model can rank most candidates correctly while misjudg-

ing those retained for further search. We therefore introduce planning-aligned diagnostics that assess selected elites using environment-realized outcomes on shared candidate sequences. In controlled Cube experiments, elite regret is more closely associated with closed-loop success than factual prediction error or whole-bank ranking. Across five simulation environments, AD-WM improves mean success over matched LeWM in four environments. It also improves zero-shot transfer to a real robot without lab-specific adaptation. Our contributions are threefold:

- We identify a mismatch between factual prediction accuracy and counterfactual action comparison in JEPA-based MPC. Our elite-selection diagnostics better reflect closed-loop success in controlled Cube experiments.
- We introduce AD-WM, which combines residual latent prediction with predictor-level action recovery to preserve action information without modifying the MPC planner.
- We evaluate AD-WM across five simulation environments and a real Franka robot. Baseline comparisons, controlled ablations, and planning diagnostics establish its control benefits and examine the factors supporting transfer.

II. RELATED WORK

Latent World Models for Control. Learned dynamics support model-based control by predicting the consequences of actions before execution [14], [15]. Existing approaches use these predictions for latent-space planning, behavior learning through imagined rollouts, or visual foresight [1], [2], [5], [16], [17]. More recent work explores Transformer and diffusion architectures for world modeling [18], [19]. Robot world models also investigate action-conditioned prediction and counterfactual behavior [20]. These approaches differ in how their predictions guide control. Our work focuses on reward-free, image-goal MPC learned from offline data. In this setting, predicted latent transitions directly determine how the planner compares candidate action sequences. We therefore study whether these transitions preserve the action-dependent differences needed for selection.

Feature-predictive world models. Joint-embedding predictive architectures (JEPAs) predict future representations without explicit observation reconstruction [7], [8]. DINO-WM, PLDM, LeWM, and V-JEPA 2 use learned or pretrained visual features for goal-directed planning [9]–[12]. LeWM is our closest end-to-end simulation baseline, while V-JEPA 2-AC provides pretrained dynamics for our real-robot study. JEPA’s emphasis on slowly varying features [21] raises the question of whether predictions adequately capture smaller action-dependent changes needed for control. Recent extensions address complementary limitations of JEPA-based control. Fast-LeWM improves rollout efficiency, while Sub-JEPA regularizes representations for stable end-to-end learning [22], [23]. INTACT and Qantara introduce action-generation and control interfaces beyond standard CEM planning [24], [25]. Delta-JEPA promotes action sensitivity by recovering actions from observed latent displacements [26].

Our normalized recovery regularizes predicted transitions, as does the default inverse objective. Combined with residual prediction, these objectives shape latent dynamics for counterfactual action comparison without modifying the MPC planner.

Action-aware representation learning. Inverse dynamics provides a training signal for learning action-relevant representations from observed state transitions [27], [28]. Related work studies Markov state abstractions and action-sufficient representations for control [29], [30]. These approaches motivate retaining information that is useful for predicting and influencing environment dynamics. We apply this principle directly at the predictor level. Our inverse dynamics and normalized action-recovery objectives operate on the current and predicted next representations. Their gradients therefore constrain the model-generated transitions that MPC rolls out and compares during planning. This complements factual prediction supervision with an explicit objective for preserving action information in predicted transitions.

III. METHOD

Fig. 2 illustrates the training and planning pipeline of AD-WM. During training, residual latent prediction and action-recovery objectives jointly shape action-discriminative transitions. At test time, MPC uses the learned dynamics to compare candidate action sequences, with the auxiliary heads discarded. We first formulate the task, then detail the model, training objectives, and planning procedure. Finally, we introduce diagnostics for evaluating candidate selection.

A. Problem Formulation

Given an offline dataset $\mathcal{D} = \{(o_t, a_t, o_{t+1})\}$ of images and continuous actions, we learn an encoder $z_t = e_\phi(o_t)$ and latent dynamics $\hat{z}_{t+1} = F_\theta(z_t, a_t)$. At test time, the agent receives a current image and a goal image g , encoded as $z_g = e_\phi(g)$. MPC evaluates candidate action sequences $a = a_{t:t+H-1}$ using the predicted terminal cost $\hat{c}(a) = \|\hat{z}_{t+H}(z_t, a) - z_g\|_2^2$. It executes the first action block and replans without parameter updates. Low factual prediction error alone does not ensure useful action comparison. For transitions $z_{t+1} = z_t + \delta_t$ with small increments, an action-independent predictor $F_0(z, a) = z$ incurs only $\mathbb{E}\|\delta_t\|_2^2$ expected error. Yet it assigns all action sequences the same terminal cost. We therefore seek latent dynamics that preserve action-dependent differences needed for planning.

B. Residual Latent Prediction

Successive visual representations often share substantial state information. We let the current representation carry this shared information and train the predictor to estimate the remaining change. Given a learned action embedding $e_t = \psi_\rho(a_t)$, AD-WM predicts:

$$\Delta \hat{z}_t = f_\theta(z_t, e_t), \quad \hat{z}_{t+1} = z_t + \Delta \hat{z}_t. \quad (1)$$

The prediction is supervised by the encoded successor:

$$\mathcal{L}_{\text{pred}} = \|\hat{z}_{t+1} - z_{t+1}\|_2^2, \quad z_{t+1} = e_\phi(o_{t+1}). \quad (2)$$

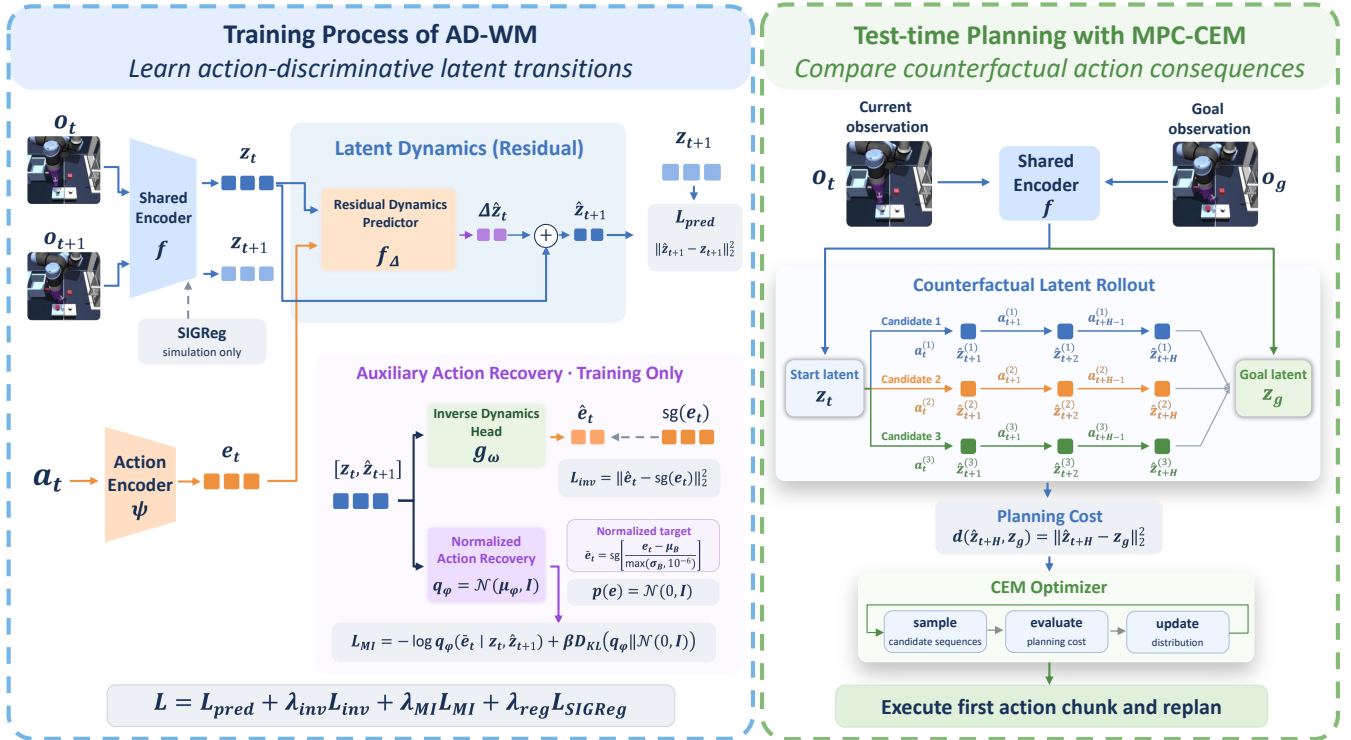


Fig. 2: **Training and planning with AD-WM.** Left: residual dynamics predicts increments supervised by encoded successors. Inv and MI encourage action recovery from predicted transitions. Simulation jointly trains the encoder and predictor with SIGReg. Robot post-training freezes the encoder and omits SIGReg. Right: CEM scores rollouts by terminal cost $\|\hat{z}_{t+H} - z_g\|_2^2$, executes the first action block, and replans. Auxiliary heads are discarded at deployment.

This equivalently matches $\Delta \hat{z}_t$ to the encoded displacement $z_{t+1} - z_t$. Absolute and residual prediction share the same optimum in an unconstrained function class. The residual parameterization changes the learning bias by explicitly modeling local latent changes. However, it does not itself require these changes to retain action information. We address this through predictor-level action recovery.

C. Predictor-Level Action Recovery

We encourage actions to be recoverable from the current and predicted next representations. Applying recovery to model-generated transitions directly shapes the dynamics used by MPC. Prediction supervision anchors these transitions to observed outcomes, while recovery discourages the predictor from ignoring action information.

Inverse dynamics. The default inverse head recovers the action embedding from predicted transition endpoints:

$$\hat{e}_t = g_\omega(z_t, \hat{z}_{t+1}), \quad \mathcal{L}_{\text{inv}} = \|\hat{e}_t - \text{sg}(e_t)\|_2^2. \quad (3)$$

The operator sg detaches the recovery target. The predicted endpoint remains differentiable, allowing auxiliary gradients to train the predictor and action encoder ψ_ρ through $\hat{z}_{t+1}$. Thus, recovery constrains generated transitions rather than only observed state representations. Section IV-C examines alternative inverse inputs.

Normalized action recovery. Inv operates on learned action embeddings at their current scale. Normalized recovery uses standardized targets and a Gaussian recovery objective motivated by conditional mutual information (MI).

We normalize embeddings componentwise as $\bar{e}_t = \text{sg}[(e_t - \mu_B) / \max(\sigma_B, 10^{-6})]$, where μ_B and σ_B are the batch mean and standard deviation. This controls target scale for likelihood training with fixed unit covariance.

A separate head receives $[z_t, \hat{z}_{t+1}]$ and defines $q_\eta(\cdot | z_t, \hat{z}_{t+1}) = \mathcal{N}(\mu_\eta, I)$. Using a standard Gaussian as a fixed regularization reference, we minimize:

$$\mathcal{L}_{\text{MI}} = -\log q_\eta(\bar{e}_t | z_t, \hat{z}_{t+1}) + \beta D_{\text{KL}}(q_\eta(\cdot | z_t, \hat{z}_{t+1}) \| \mathcal{N}(0, I)). \quad (4)$$

With unit covariance, this reduces, up to an additive constant, to:

$$\frac{1}{2} \|\bar{e}_t - \mu_\eta\|_2^2 + \frac{\beta}{2} \|\mu_\eta\|_2^2.$$

The first term recovers standardized action embeddings. The second shrinks the predicted mean toward zero. For fixed state/action representations and target normalization, the recovery log-likelihood is the variational term in a Barber-Agakov lower bound on $I(\bar{e}_t; \hat{z}_{t+1} | z_t)$ [31]. This motivates the recovery term, with additional KL regularization during joint training.

D. Training and Planning

In simulation, the encoder and predictor are trained jointly with:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \mathcal{L}_{\text{sig}} + \lambda_{\text{inv}} \mathcal{L}_{\text{inv}} + \lambda_{\text{MI}} \mathcal{L}_{\text{MI}}. \quad (5)$$

Prediction supervision matches observed outcomes, while the recovery objectives encourage predicted transitions to retain

action information. We use the same SIGReg representation regularizer as LeWM [11]. Robot post-training freezes the encoder and omits SIGReg (Section IV-E).

At deployment, both auxiliary heads are discarded. MPC embeds candidate actions with ψ_ρ and recursively applies Eq. (1). CEM retains the sequences with lowest predicted terminal cost and refits its sampling distribution to this elite set. After search, the agent executes the first action block and replans from the new observation. The encoder architecture and CEM procedure remain unchanged.

E. Planning-Facing Diagnostics

To assess whether predicted transitions support useful action selection, we evaluate global ranking and elite quality on a shared candidate bank $\mathcal{A} = \{a^{(i)}\}_{i=1}^N$. For each sequence, $\hat{c}(a)$ is its predicted terminal cost. Its realized cost $c^*(a)$ is obtained by executing the sequence from the same initial environment state and encoding the final image. These realized costs are used only for simulation diagnostics, outside training and MPC. Each model uses its own encoder for both costs, so the metrics assess alignment within its planning space.

Global ranking. Counterfactual action discriminability (CAD) measures agreement across the full candidate bank:

$$\text{CAD}(z_t) = \rho_S \left(\{\hat{c}(a^{(i)})\}_{i=1}^N, \{c^*(a^{(i)})\}_{i=1}^N \right), \quad (6)$$

where ρ_S is Spearman rank correlation. This includes candidates outside the elite set.

Elite quality. Let $E_k(c)$ denote the k lowest-cost candidates under c . Define the realized cost range $D_{\mathcal{A}} = \max_{a \in \mathcal{A}} c^*(a) - \min_{a \in \mathcal{A}} c^*(a)$, floored at 10^{-8} . Normalized *best-in-elite regret* is:

$$R_k = \frac{\min_{a \in E_k(\hat{c})} c^*(a) - \min_{a \in \mathcal{A}} c^*(a)}{D_{\mathcal{A}}}. \quad (7)$$

Low R_k indicates that the predicted elite set retains at least one near-best candidate. Since CEM refits its proposal using all elites, we also define *elite-mean regret*:

$$\bar{R}_k = \frac{\sum_{a \in E_k(\hat{c})} c^*(a) - \sum_{a \in E_k(c^*)} c^*(a)}{k D_{\mathcal{A}}}. \quad (8)$$

This compares the mean realized cost of predicted elites with that of the realized top- k set. Both regrets normalize selection penalties by the bank’s realized cost range. They characterize candidate selection, while closed-loop success remains the final control outcome.

IV. EXPERIMENTS

We organize our experiments around four questions: **Q1:** How well does AD-WM perform across simulation tasks and configuration shifts? **Q2:** How do residual prediction and action recovery contribute to control performance? **Q3:** Which prediction and selection metrics best reflect closed-loop success? **Q4:** Can AD-WM transfer to a real robot without lab-specific adaptation?

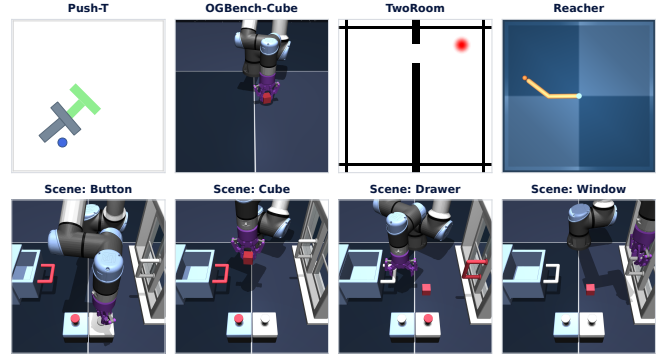


Fig. 3: The simulation environments.

A. Experimental Setup

Benchmarks and Baselines. We follow LeWM’s simulation environment selection [11], evaluating on Cube, Reacher, TwoRoom, and PushT, and additionally include Scene from OGBench [32]. As shown in Fig. 3, these environments cover robotic manipulation, reaching, navigation, and planar pushing. Cube serves as the primary benchmark for controlled ablations and planning diagnostics. We retain the original evaluation protocols for cross-environment comparisons, except for Scene, and introduce additional hard-start protocols on Cube. Our main baseline is a matched LeWM reproduction [11]. We also evaluate released Cube checkpoints from Fast-LeWM [22], Sub-JEPA [23], and INTACT [24], retaining their native inference interfaces. Controlled ablations share images, encoder size, training budget, and MPC settings, varying only transition parameterization and auxiliary training choices.

Evaluation protocols. Cube includes Original, which samples dataset states, and hard-start protocols P00–P04. Hard starts select tabletop states without gripper contact, with minimum goal distance 0.05 and end-effector–cube distance 0.02. P00–P04 apply cube xy perturbations of radii 0.00/0.01/0.02/0.03/0.04, clipped to $x \in [0.30, 0.55]$ and $y \in [-0.30, 0.30]$. P00 therefore differs from Original even without perturbation. Each protocol uses 50 episodes per checkpoint with evaluation seed 42. Hard-start success (HS) averages P00–P04. Unless stated otherwise, Cube results are means and population standard deviations (SDs) over seeds 3072, 4096, and 6144. Reacher, TwoRoom, and PushT use original protocols. Scene uses 200 hard-start episodes per checkpoint, balanced across Button, Cube, Drawer, and Window. Starts differ from goals in one component, with change-detection tolerance 0.04 and minimum target displacement and end-effector–target distance 0.08 for geometric filters.

We report success as percentages and factual latent mean squared error (MSE) at one step and averaged over the first five rollout steps (local MSE). Counterfactual action discriminability (CAD) measures whole-bank ranking agreement; R_{30} and $\bar{R}_{30}$ denote normalized best-in-elite and elite-mean regret for 30 selected candidates (Section III-E). We use ρ for Spearman rank correlation.

Training details. Our simulation setup follows LeWM [11], using 224×224 images, training/evaluation

TABLE I: **Cube success (%) on matched starts.** Fast-LeWM and Sub-JEPA use one checkpoint. Others report mean $\pm$ population SD over three. External methods retain native inference. Bold marks the highest mean.

Method	Inference	Original	P00	P01	P02	P03	P04
LeWM	CEM	73.3 $\pm$ 2.5	8.0 $\pm$ 2.8	4.7 $\pm$ 0.9	4.7 $\pm$ 1.9	1.3 $\pm$ 1.9	0.0 $\pm$ 0.0
Fast-LeWM [22]	CEM	80.0	24.0	20.0	10.0	2.0	0.0
Sub-JEPA [23]	CEM	78.0	34.0	20.0	10.0	10.0	10.0
INTACT [24]	Pure CEM	74.7 $\pm$ 0.9	20.0 $\pm$ 3.3	21.3 $\pm$ 9.3	13.3 $\pm$ 2.5	6.7 $\pm$ 1.9	3.3 $\pm$ 2.5
INTACT	Actor-CEM	94.7 $\pm$ 2.5	78.7 $\pm$ 3.4	72.0 $\pm$ 5.7	56.0 $\pm$ 4.3	33.3 $\pm$ 12.4	10.7 $\pm$ 2.5
INTACT	Direct	98.7$\pm$1.9	99.3$\pm$0.9	99.3$\pm$0.9	88.7$\pm$1.9	35.3 $\pm$ 3.4	10.0 $\pm$ 0.0
AD-WM	CEM	90.7 $\pm$ 3.4	74.0 $\pm$ 2.8	68.0 $\pm$ 4.3	56.0 $\pm$ 1.6	36.7$\pm$5.2	25.3$\pm$3.4

histories of 3/1, and frame skip 5. The trainable ViT-tiny encoder [33] has latent dimension 192. The predictor has depth 6, 16 heads, and MLP dimension 2048. Inv and MI heads use 1024-hidden-unit MLPs with BatchNorm. Training uses bf16, 10 epochs, AdamW, batch size 128, learning rate 5×10^{-5} , weight decay 10^{-3} , and $\lambda_{\text{sig}} = 0.09$. All simulation environments use $\lambda_{\text{inv}} = 0.1$, $\lambda_{\text{MI}} = 0.01$, and $\beta = 0.01$, except Scene, where $\lambda_{\text{MI}} = 10^{-4}$. Primary weights were specified before sensitivity analysis.

Planning details. Matched comparisons use CEM [13] with 300 candidates, 30 elites, 30 iterations, horizon 5, receding horizon 5, and action block 5. Actions are clipped to environment bounds and scored by terminal latent distance. Cube and Scene use goal offset 25 and a 50-step budget. Sub-JEPA and INTACT CEM modes use the same candidate count, elite count, iterations, horizon, and action block. Fast-LeWM uses horizon 1 and action block 25. INTACT Direct performs no search. External Cube comparisons share starts, episode counts, goal offset, and budget, but do not control training and inference as the LeWM-AD-WM pair does.

B. Simulation Results

Main results. To answer Q1, Table I compares methods on identical Cube starts. AD-WM outperforms matched LeWM across all protocols. INTACT Direct performs best from Original through P02, while its Actor-CEM mode improves over Pure CEM through P03. On P03, INTACT Direct, Actor-CEM, and AD-WM have similar means relative to checkpoint variation. On P04, AD-WM reaches 25.3%, compared with 10.0% and 10.7% for Direct and Actor-CEM. These results indicate stronger performance under larger perturbations, but differences in training and inference prevent attributing the crossover to search strategy alone.

Configuration shift. P00–P01 serve as near-distribution controls, while P02–P04 probe configuration tails within empirical cube xy support. We quantify shift using Euclidean nearest-neighbor distances to training states from other episodes. Joint coordinates concatenate cube xy and end-effector xyz , while relative coordinates use end-effector-minus-cube xyz . Reference 95th-percentile thresholds use 20,000 tabletop/non-contact training states per seed. For P02–P04, 2.0%/38.7%/72.0% exceed the joint threshold, and 52.7%/80.7%/95.3% exceed the relative threshold.

Cross-environment performance. Fig. 4(a) shows higher mean success than reproduced LeWM in four of five environments: Cube (73.3% $\rightarrow$ 90.7%), Reacher (76.7% $\rightarrow$ 83.3%),

TABLE II: **Cube ablations (% , three seeds).** A/B show mean $\pm$ population SD, C/D show means. HS averages P00–P04. Blue marks proposed/default settings in A/C/D. Bold indicates column-best means in A and row-best means in C/D.

A. Component contributions						
Variant		Original $\uparrow$			HS $\uparrow$	
LeWM		73.3 $\pm$ 2.5			3.7 $\pm$ 1.4	
Abs. + Inv + MI		83.3 $\pm$ 0.9			14.4 $\pm$ 2.9	
Res		82.7 $\pm$ 2.5			34.7 $\pm$ 1.6	
Res + Inv		83.3 $\pm$ 1.9			37.1 $\pm$ 4.7	
Res + MI		89.3 $\pm$ 3.8			54.7 $\pm$ 3.0	
AD-WM		90.7 $\pm$ 3.4			52.0 $\pm$ 3.1	
B. MI gains across inverse inputs (HS)						
Input		No MI			MI = 0.01	
Predicted endpoints		37.1 $\pm$ 4.7			52.0 $\pm$ 3.1	
Encoded endpoints		45.1 $\pm$ 4.4			60.7 $\pm$ 0.4	
Predicted increment		39.9 $\pm$ 2.1			60.3 $\pm$ 3.7	
C. MI weight ($\lambda_{\text{inv}} = 0.1$)						
λ_{MI}	0	.005	.01	.015	.03	.05
Original	83.3	89.3	90.7	90.7	91.3	88.0
HS	37.1	50.9	52.0	60.4	65.2	61.5
D. Inv weight ($\lambda_{\text{MI}} = 0.01$)						
λ_{inv}	0	.05		.10		.20
Original	89.3	90.0		90.7		92.0
HS	54.7	57.2		52.0		57.5

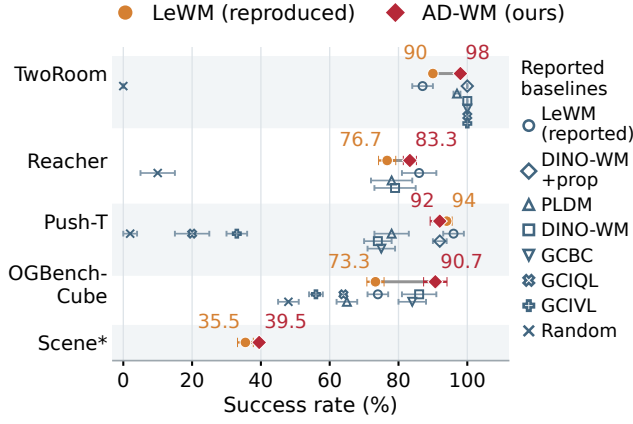
MI-enabled variants in A use $\lambda_{\text{MI}} = 0.01$. In B, $\lambda_{\text{inv}} = 0.1$ and MI always uses predicted endpoints. Encoded endpoints remove only the direct Inv gradient through the predicted successor.

TwoRoom (90% $\rightarrow$ 98%), and Scene (35.5% $\rightarrow$ 39.5%). PushT decreases from 94% to 92%. All environments except Scene use original protocols. Reported external results are shown separately, including LeWM’s reported 86% on Reacher. Our improvement claim concerns the matched reproduction. Scene gains concentrate in Drawer (63.3% $\rightarrow$ 69.3%) and Window (46.7% $\rightarrow$ 55.3%), both improving in all three matched seed comparisons. Button is similar and Cube remains difficult. The overall gain remains unresolved by the paired seed test ($p = 0.13$).

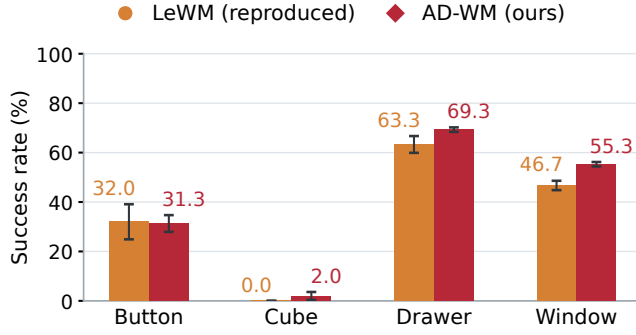
C. Ablation Studies

Table II addresses Q2 through component ablations (A), MI gains across inverse inputs (B), and post-hoc weight sensitivity (C/D).

Component contributions. Panel A identifies residual prediction and MI as the largest contributors. LeWM achieves 3.7% HS. Adding Inv+MI to absolute prediction



(a) Cross-environment performance.



(b) Scene subtask performance.

Fig. 4: **Cross-environment and subtask performance.** (a) Matched LeWM and AD-WM results, with reported baselines shown separately. Scene (*) uses balanced hard starts, while other environments use original protocols. (b) Scene results over three seeds and 50 episodes per subtask, with $\lambda_{\text{MI}} = 10^{-4}$. Error bars show source SDs in (a) and population SDs in (b).

raises this to 14.4%, while residual prediction alone reaches 34.7%. Res+Inv achieves 37.1%, Res+MI 54.7%, and AD-WM 52.0%. Inv gives a smaller mean gain over Res and does not improve upon Res+MI at its default weight.

MI gains across inverse inputs. Panel B varies Inv inputs while MI always uses predicted endpoints. MI adds 14.9/15.6/20.4 mean HS points across predicted endpoints, encoded endpoints, and predicted increments. The latter two reach 60.7/60.3%, versus 52.0% for the default; predicted increments still backpropagate Inv through the predictor. Thus, B tests Inv-input robustness, not MI-input robustness.

Weight sensitivity. With Inv fixed at 0.1, all tested nonzero MI weights (0.005–0.05) yield higher mean HS: 50.9–65.2% versus 37.1% (Panel C). MI peaks at 0.03; the pre-specified 0.01 remains primary. With MI fixed at 0.01, Inv weights 0.05/0.20 exceed the no-Inv mean, but the default 0.10 does not (Panel D). Inv’s effect is smaller and weight-dependent.

D. Planning Diagnostics

Evaluation setup. To answer Q3, Table III compares prediction error, candidate selection, and HS. Factual errors use 61,999 held-out clips with three-frame context. Candidate selection evaluates five default-weight variants over three seeds on 64 shared cases with 300 candidates each. Banks contain four reference sequences (expert, zero, negated-expert, and reversed-expert) and 99/99/98 near/medium/far perturbations. Actions and simulator endpoints are shared, while costs use each model’s encoder. The default elite size $k = 30$ matches CEM, with $k = 15, 60$ testing robustness.

Prediction and selection quality. LeWM has the lowest factual MSE but also the lowest HS; AD-WM reaches 52.0% HS despite the highest MSE. These errors depend on representation and scale. Across 15 model–seed observations, HS correlates with CAD at $\rho = -0.399$ and with negative R_{30} and $\bar{R}_{30}$ at 0.863 and 0.810, respectively. The regret associations persist at $k = 15, 60$. Hierarchical paired bootstrap over seeds and cases (20,000 resamples) gives successive R_{30} reductions along LeWM–Res–Res+Inv–AD-WM of 0.028 [0.014, 0.043], 0.002 [−0.003, 0.009], and 0.014 [0.009, 0.023] (95% CIs). The Inv-only increment remains unresolved; Res+MI and AD-WM have similar mean regret. From LeWM to AD-WM, realized-best candidate retention falls from 0.198 to 0.135; a nearly tied alternative can still yield low regret. These associations concern fixed-bank selection, not adaptive CEM trajectories.

Local dynamics and search geometry. We examine the first five steps of 25-step rollouts and 20 CEM landscapes on a 31×31 grid along the first two principal directions of the final elites. From LeWM to Res, latent-increment MSE decreases from 0.0067 to 0.0055, and the fraction of consecutive predicted increments with negative cosine similarity falls from 0.242 to 0.146. From LeWM to AD-WM, the distance between the CEM center and the lowest-cost grid point decreases from 0.827 to 0.130, while mean elite-to-center distance falls from 0.448 to 0.082. These results are consistent with more stable local updates and more concentrated search.

E. Real-Robot Transfer

Setup and post-training. To address Q4, we evaluate transfer on the Franka setup in Fig. 5(a). We compare AD-WM with V-JEPA 2-AC [12] using the same frozen V-JEPA 2 ViT-G encoder, filtered *cadene/droid.1.0.1* data [34], predictor architecture, training schedule, CEM planner, clipping, and deployment stack. Only transition parameterization and auxiliary objectives differ. No laboratory images or demonstrations are used for adaptation. The predictor has dimension 1024, depth 24, and 16 heads. Post-training uses four A800 GPUs, bf16, 315 epochs, 300 iterations per epoch, batch size 8 per GPU, and peak learning rate 4.25×10^{-4} . Auxiliary weights are $\lambda_{\text{inv}} = 0.1$, $\lambda_{\text{MI}} = 5 \times 10^{-5}$, and $\beta = 0.1$, without tuning on Franka evaluation results. Actions are DROID-style 7D Cartesian/gripper deltas. Each replan predicts one latent transition using 800 CEM samples, 10 elites, and 10 iterations.

TABLE III: **Prediction, selection, and control metrics on Cube (mean $\pm$ population SD, three seeds).** Local MSE averages five rollout steps. MSE values are multiplied by 10^3 . Shared banks use $N = 300$ and $k = 30$. Blue marks AD-WM. Bold marks the best displayed mean per metric, including ties.

Variant	Factual prediction		Candidate selection			Control
	One-step MSE $\downarrow$	Local MSE $\downarrow$	CAD $\uparrow$	$R_{30} \downarrow$	$\bar{R}_{30} \downarrow$	HS (%) $\uparrow$
LeWM	2.72 $\pm$ 0.06	10.36 $\pm$ 0.48	0.460 $\pm$ 0.010	0.074 $\pm$ 0.006	0.298 $\pm$ 0.010	3.7 $\pm$ 1.4
Res	3.61 $\pm$ 0.20	11.17 $\pm$ 0.61	0.469 $\pm$ 0.005	0.046 $\pm$ 0.003	0.250 $\pm$ 0.009	34.7 $\pm$ 1.6
Res + Inv	3.43 $\pm$ 0.11	11.00 $\pm$ 0.28	0.470 $\pm$ 0.005	0.043 $\pm$ 0.002	0.245 $\pm$ 0.005	37.1 $\pm$ 4.7
Res + MI	4.20 $\pm$ 0.06	15.17 $\pm$ 0.04	0.454 $\pm$ 0.002	0.029 $\pm$ 0.002	0.228 $\pm$ 0.002	54.7 $\pm$ 3.0
AD-WM	4.27 $\pm$ 0.10	15.77 $\pm$ 0.20	0.448 $\pm$ 0.011	0.029 $\pm$ 0.001	0.229 $\pm$ 0.005	52.0 $\pm$ 3.1

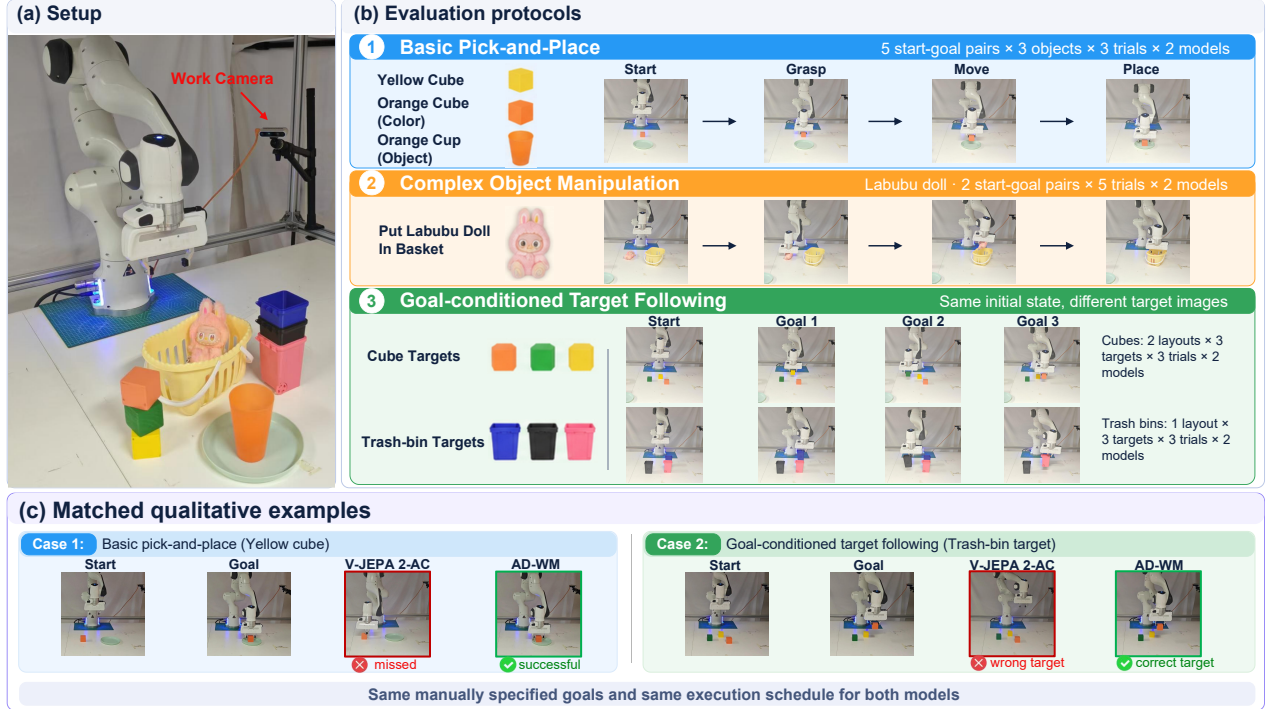


Fig. 5: **Real-robot evaluation.** (a) Franka setup with a single camera. (b) Basic pick-and-place, complex-object, and specified-target protocols use 45, 10, and 27 trials per model. (c) Matched examples illustrate grasping and target selection. Both models share image goals, execution stages, gripper handling, and action clipping.

Evaluation protocol. A single side/rear RealSense camera provides eight frames at 4 fps with a 256×256 crop. MPC follows manually supplied grasp, move, and place image goals, with 10/10/4 execution steps. Cartesian deltas are clipped to norm 0.075 and converted to absolute base-frame targets. Gripper commands are thresholded, with forced closing during grasp and opening during place. This evaluates short-horizon MPC within a structured subgoal pipeline.

Each model receives 45 basic pick-and-place trials (three objects, five start-goal settings, three repeats), 10 complex-object trials, and 27 specified-target trials. The total is 82 trials per model and 164 overall. No trials are excluded, and safety stops count as failures. Evaluation uses separate, non-randomized model blocks.

Results. Fig. 5(b) illustrates three protocols: pick-and-place across object colors and shapes, placing a Labubu doll in a basket, and selecting objects through different goal images. Table IV shows higher basic pick-and-place success, from 42.2% to 71.1%. All three objects improve

TABLE IV: **Zero-shot Franka results (count/total).** Abnormal motion counts adverse events. Bold marks better results.

Protocol	Metric	V-JEPA 2-AC	AD-WM
Basic	Success $\uparrow$	19/45	32/45
Complex	Success $\uparrow$	2/10	5/10
	Abnormal motion $\downarrow$	6/10	2/10
Target	Target moved $\uparrow$	14/27	21/27
	Lift-and-place $\uparrow$	9/27	17/27

(33.3% $\rightarrow$ 60.0%, 46.7% $\rightarrow$ 73.3%, and 46.7% $\rightarrow$ 80.0%). Complex-object and specified-target results favor AD-WM, though smaller samples limit precision. In Fig. 5(c), AD-WM grasps the cube and selects the requested trash bin, while V-JEPA 2-AC misses the grasp or moves the wrong target. Abnormal motion denotes uncontrolled or implausible movement. Specified-target metrics distinguish moving the requested object from completing lift-and-place. These results support transfer within the shared deployment pipeline.

V. CONCLUSION AND LIMITATIONS

We present AD-WM, an action-discriminative world model for counterfactual MPC. It combines residual prediction with predictor-level action recovery to preserve action information without changing the planner. Experiments show gains over matched LeWM in four of five simulation environments and transfer to Franka without lab-specific adaptation. Cube ablations identify residual prediction and MI as the largest contributors, while Inv has a smaller, weight-dependent effect on mean success. Cube diagnostics show that elite regret tracks success more closely than factual error or whole-bank ranking.

Diagnostics remain limited to Cube, and Scene’s gain is unresolved over three seeds. External comparisons differ in inference and checkpoint counts. Simulation and robot post-training use different auxiliary weights. Robot evaluation uses manual subgoals and non-randomized trials with one site, camera, and backbone. Future work examines broader settings and reduces reliance on manual subgoals.

REFERENCES

- [1] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, “Learning latent dynamics for planning from pixels,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 2555–2565.
- [2] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” in *International Conference on Learning Representations*, 2020.
- [3] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba, “Mastering atari with discrete world models,” in *International Conference on Learning Representations*, 2021.
- [4] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, vol. 640, pp. 647–653, 2025.
- [5] N. Hansen and X. Wang, “Temporal difference learning for model predictive control,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 8383–8396.
- [6] N. Hansen, H. Su, and X. Wang, “TD-MPC2: Scalable, robust world models for continuous control,” in *International Conference on Learning Representations*, 2024.
- [7] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 15 619–15 629.
- [8] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas, “Revisiting feature prediction for learning visual representations from video,” *Transactions on Machine Learning Research*, 2024.
- [9] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, “DINO-WM: World models on pre-trained visual features enable zero-shot planning,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 79 115–79 135.
- [10] V. Sobal, W. Zhang, K. Cho, R. Balestriero, T. G. J. Rudner, and Y. LeCun, “Learning from reward-free offline data: A case for planning with latent dynamics models,” 2025.
- [11] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, “LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels,” 2026.
- [12] M. Assran, A. Bardes, D. Fan *et al.*, “V-JEPA 2: Self-supervised video models enable understanding, prediction and planning,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.09985>
- [13] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.
- [14] D. Ha and J. Schmidhuber, “World models,” *arXiv preprint arXiv:1803.10122*, 2018. [Online]. Available: <https://arxiv.org/abs/1803.10122>
- [15] R. S. Sutton, “Dyna, an integrated architecture for learning, planning, and reacting,” *ACM SIGART Bulletin*, vol. 2, no. 4, pp. 160–163, 1991.
- [16] M. Watter, J. T. Springenberg, J. Boedecker, and M. Riedmiller, “Embed to control: A locally linear latent dynamics model for control from raw images,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [17] C. Finn and S. Levine, “Deep visual foresight for planning robot motion,” in *IEEE International Conference on Robotics and Automation*, 2017, pp. 2786–2793.
- [18] V. Micheli, E. Alonso, and F. Fleuret, “Transformers are sample-efficient world models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=vhFu1Ac0xb>
- [19] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret, “Diffusion for world modeling: Visual details matter in atari,” in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=NadTwTODgC>
- [20] S. Gao, W. Liang, K. Zheng, A. Malik, S. Ye, S. Yu, W.-C. Tseng, Y. Dong, K. Mo, C.-H. Lin, Q. Ma, S. Nah, L. Magne, J. Xiang, Y. Xie, R. Zheng, D. Niu, Y. L. Tan, K. R. Zentner, G. Kurian, S. Indupuru, P. Jannaty, J. Gu, J. Zhang, J. Malik, P. Abbeel, M.-Y. Liu, Y. Zhu, J. Jang, and L. J. Fan, “DreamDojo: A generalist robot world model from large-scale human videos,” 2026.
- [21] V. Sobal, S. V. Jyothir, S. Jalagam, N. Carion, K. Cho, and Y. LeCun, “Joint embedding predictive architectures focus on slow features,” 2022.
- [22] Y. Gao and X. Xu, “Fast LeWorldModel,” *arXiv preprint arXiv:2606.26217*, 2026.
- [23] K. Zhao, D. Nie, Y. Lin, Z. Luo, Y. Gu, D.-P. Fan, and D. Zeng, “Sub-JEPA: Subspace gaussian regularization for stable end-to-end world models,” 2026.
- [24] J. Sun, H. Zhao, and G. Zhang, “INTACT: Isomorphic intent-to-action learning for search-free world models,” 2026.
- [25] R. Rakhimov, G. Bredis, Y. Maksyuta, and D. Gavrilo, “Qantara: Bridge-flow training for multi-paradigm JEPA control,” 2026.
- [26] Z. Zhang, Y. Wang, Z. Guan, Y. Yang, B. Shi, T. Zong, H. Yi, G. Chao, X. Chen, T. Yang, C. Bao, T. Yu, J. Zhou, and J. Xu, “Delta-JEPA: Learning action-sensitive world models via latent difference decoding,” 2026.
- [27] P. Agrawal, A. V. Nair, P. Abbeel, J. Malik, and S. Levine, “Learning to poke by poking: Experiential learning of intuitive physics,” in *Advances in Neural Information Processing Systems*, vol. 29, 2016.
- [28] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, “Curiosity-driven exploration by self-supervised prediction,” in *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- [29] C. Allen, N. Parikh, O. Gottesman, and G. Konidaris, “Learning markov state abstractions for deep reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 8229–8241.
- [30] B. Huang, C. Lu, L. Leqi, J. M. Hernández-Lobato, C. Glymour, B. Schölkopf, and K. Zhang, “Action-sufficient state representation learning for control with structural constraints,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 9260–9279.
- [31] D. Barber and F. V. Agakov, “Information maximization in noisy channels: A variational approach,” in *Advances in Neural Information Processing Systems*, vol. 16, 2003.
- [32] S. Park, K. Frans, B. Eysenbach, and S. Levine, “OGBench: Benchmarking offline goal-conditioned reinforcement learning,” in *International Conference on Learning Representations*, 2025.
- [33] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [34] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” in *Robotics: Science and Systems*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.12945>